Stop selling air

Predict tomorrow’s empty shelves tonight — and fix them before customers notice

For: COO / VP Supply Chain (executive sponsor) · Director of Replenishment & Store Operations (domain owner)

The ask: a 12-week pilot in 10 stores to recover **~$0.48M of lost sales a year** (≈ **+1.6 pts on-shelf availability**), scaling to **~$6.8M per $1B of sales** chain-wide.

Prototype built end-to-end on Databricks with real Walmart demand data (public M5 dataset) and simulated ERP data.

The problem: 1 in 10 shelves is empty, and we find out too late

Measured on 52 weeks of real store-item sales (10 stores, 3,049 items, to 22-May-2016):

KPI	Today	FY target (RACI-006)
On-shelf availability (OSA)	**90.4 %**	≥ 96 %
Revenue lost to stockouts	**$2.82 M / yr** (5.9 % of sales)	< 1.5 %
Stockout events, Nov-2015 → May-2016 (root-caused window)	**49,600** events, avg **6.7 days** each	—
Root cause: supplier PO arrived late	**75 %** of events, **$1.03 M** lost in 6 months	—
Root cause: "phantom" inventory (system shows stock)	**25 %** of events, **$0.34 M** lost in 6 months	—

Why it keeps happening: the warning signs (an overdue PO, cover running down, zero sales while the system shows stock) live in POS, ERP and WMS separately. Today's rule — *"alert if cover < 3 days or a PO is late"* — fires **4,800 alerts a day** and **77 % of them are false alarms**, so stores ignore it.

What we built: one nightly loop from raw data to a decision on the shelf

1. **Ingest (Lakeflow):** POS sales, ERP inventory snapshots and PO events land nightly; Auto Loader + declarative pipeline cleans them (bad rows quarantined by data-quality rules) into a governed store-item-day model.
2. **Govern (Unity Catalog):** one copy of the data; regional managers see only their region, supplier terms and costs are masked; lineage from raw file to app is automatic.
3. **Predict (ML + GenAI):** a model scores every store-item for *“out of stock in the next 3 days”*, explains why, sizes the order per our SOP, and writes a one-line instruction for the store manager.
4. **Serve (Lakebase):** the action queue is served to the app in milliseconds; store decisions are written back transactionally and flow into the next night’s run.
5. **Ask (Genie + agents):** leaders ask questions in plain English over the data *and* our SOPs/supplier contracts; an agent writes the 7 am briefing for the exec team and each region.
6. **Act (Databricks App):** *OSA Command Center* — tonight’s queue, approve/override with reason codes, briefings, Q&A.

Executive view: the money

	10-store pilot (measured base)	Per $1B of chain sales (conservative)
Lost sales today	$2.82 M / yr	$40 M (industry 4 % lost-sales rate)
Lost sales the daily alert list covers	33.8 % (vs 14.2 % for today's rule)	33.8 %
Assumed share stores actually prevent	50 %	50 %
Recovered revenue / yr	**$0.48 M** (+1.05 % of sales)	**$6.8 M**
Gross margin on recovered sales (25 %)	$0.12 M	$1.7 M
Incremental vs today's rule	**+$0.28 M**	**+$3.9 M**
On-shelf availability	90.4 % → **~92.0 %**	+1.6 pts

Plus, not counted above: fewer emergency DSD deliveries (8 % premium + $45/store/day), supplier chargebacks backed by evidence (late PO → stockout → $ lost), and store labour saved on manual gap scans.

Payback: platform run cost for the nightly loop is serverless and scales to zero; recovered margin at pilot scale covers it many times over.
(Assumptions are explicit and adjustable — see appendix.)

Domain-owner view: better alerts, fewer of them, each with an action

Back-test on **1–19 May 2016** (unseen weeks, real Walmart sales as ground truth):

	Today's rule	Stockout-risk model
Ranking quality (ROC AUC)	0.77	**0.95**
Precision of top 500 alerts / day	48 %	**96 %**
Precision of top 100 alerts / day	47 %	**99 %**
Lost revenue covered by top 500 / day	14 %	**34 %**
Alerts to review per store per day	~480 (all flagged)	**50** (prioritised by $ at risk)

Every alert comes with: **why** (top drivers, e.g. "PO 3 days overdue · 1.2 days of cover"), **what** (expedite / emergency DSD / cycle count for suspected phantom stock), **how much** (SOP order-up-to formula, case-pack rounded) and **$ at risk**. Store managers approve or override with a reason code; overrides > 50 % are routed to the Regional Manager (SOP-REP-001). Approved quantities count as on-order in tomorrow's scoring.

What leaders see at 7 am (agent-written, numbers verified)

> **Headline:** South region OSA held at 93.6 % last week, but 210 critical alerts put $2.4k of sales at risk in the next 3 days — most from two late pantry suppliers …
>
> **Recommended actions:** 1. Category buyer to escalate *Coastal Canning Co* per POL-SUP-003 (3+ days late) …

(Excerpt — full briefings in `evidence/agents/briefings.md`.)

- One executive and three regional briefings every night, written by a **tool-calling agent** that may only use governed data functions and our policy library — it cites the procedure (POL-SUP-003, GUIDE-DSD-005 …).
- Automatically **evaluated** every night: KPI figures cross-checked against source tables, policy cited, required sections present, safety — failures are flagged before anyone reads them.
- **Ask anything:** “Which suppliers caused the most stockouts and what does our escalation policy require?” — the assistant combines governed SQL (Genie) with our SOPs and contracts in one answer, and shows the SQL it ran.

Trust and control built in

- **Governed once, enforced everywhere:** row-level security by region and masks on supplier commercial terms, item cost and e-mails are attached to the tables — the same rules apply in SQL, Genie, the agents and the app. Verified with a real South store-manager identity: sees only South, sees `***RESTRICTED***` for terms and costs.
- **Entitlements are data:** onboarding a new regional manager is one row, not a code change.
- **Explainable:** every score carries its top drivers; every agent answer shows its SQL or document citation.
- **Auditable:** lineage from source file → pipeline → model → app; every store decision stored with who/when/why.
- **Data-quality gates:** duplicates, negative stock, unknown stores and incomplete PO events are dropped and counted every night (e.g. 638 bad inventory rows, 725 incomplete PO events on the first load).

Pilot plan: 12 weeks, 10 stores, three decision gates

Weeks	Scope	Success measure (gate)
1–3	Connect real POS / ERP / WMS feeds (Lakeflow Connect), tune thresholds with Replenishment	Data freshness < 6 h; alert precision ≥ 80 % on shadow run
4–8	Live in 10 stores: store managers action the queue in the app	≥ 90 % of Critical alerts actioned in 4 h; OSA **+1 pt** vs control stores
9–12	Supplier escalation + phantom-inventory cycle counts; exec briefings to leadership	Lost-sales rate –15 %; supplier on-time +2 pts; go/no-go for chain rollout

What we need: a business owner in Replenishment, 2 store managers per region as design partners, read access to POS/ERP extracts, and a control group of 10 comparable stores.

Appendix A — how the numbers are produced

- **Stockout (OOS) day:** zero sales in a streak whose probability under the item-store's own demand rate is < 1 % ($\lambda \cdot L \geq 4.6$, streak 2–28 days); longer streaks = delisted/seasonal, excluded. Inferred from **real** M5 sales.
- **Lost sales:** demand rate × OOS days × shelf price. **OSA %:** 1 – OOS store-item-days ÷ active store-item-days.
- **Impact model:** recovered = baseline lost sales × share covered by the daily top-500 list (back-test: 33.8 %) × share prevented by acting 1–3 days early (assumption: 50 %). Chain view uses the industry 4 % lost-sales rate instead of the M5 5.9 %, i.e. conservative.
- **Caveat (honest):** inventory, POs and suppliers are *simulated* from the real sales. Their signals are cleaner than a real ERP's, so the model's precision here is an upper bound; the pilot's shadow run (weeks 1–3) re-baselines it on the customer's data before any commitment.

Appendix B — platform components used

Lakeflow Spark Declarative Pipelines + Auto Loader · Lakeflow Jobs · Unity Catalog (row filters, column masks, governed tags, lineage, model registry) · MLflow (tracking, `@champion` alias, tracing, GenAI evaluation) · `ai_query` with Foundation Model APIs · Lakebase Postgres (synced tables, transactional write-back, UC catalog) · Genie Agent · Agent Bricks Knowledge Assistant + Supervisor Agent · Databricks Apps · Declarative Automation Bundles.